

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: MLA0301
COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

***CO1:** Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 50%

Dr G.A. SENTHIL

QUESTIONS: 10

Total Marks:50

Annexure A

Constraint - Based Problem-Solving

Duration: 2Hrs

- 1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.**

Concept

Grid World is a Reinforcement Learning environment where an agent moves in a grid to reach a goal while avoiding obstacles. The agent learns the optimal path by receiving rewards or penalties.

State Space

- Each cell in the grid represents a state.
- Example: A 5×5 grid has **25 states**.

Action Space

- Up
- Down
- Left
- Right

Reward Function

- Goal reached: **+10**

- Hit obstacle: **-10**
- Normal move: **-1**

Constraints

- Agent cannot move outside the grid.
- Obstacles cannot be crossed.
- Goal must be reached with minimum steps.

Python Program

```
import gym

env = gym.make("FrozenLake-v1", is_slippery=False)

state = env.reset()

done = False

while not done:
    action = env.action_space.sample()
    state, reward, done, truncated, info = env.step(action)
    print("State:", state, "Reward:", reward)
```

Sample Output

```
State: 1 Reward: 0
State: 2 Reward: 0
State: 6 Reward: 0
State: 15 Reward: 1
```

Explanation

The agent moves randomly in the Grid World until it reaches the goal. Rewards help the agent learn the optimal path while avoiding obstacles.

OUTPUT:



```
import gymnasium as gym

env = gym.make("FrozenLake-v1", is_slippery=False)

state, info = env.reset()

terminated = False
truncated = False

while not (terminated or truncated):
    action = env.action_space.sample()
    state, reward, terminated, truncated, info = env.step(action)
    print("State:", state, "Reward:", reward)

... State: 0 Reward: 0
State: 0 Reward: 0
State: 0 Reward: 0
State: 0 Reward: 0
State: 0 Reward: 0
State: 1 Reward: 0
State: 2 Reward: 0
State: 6 Reward: 0
State: 5 Reward: 0
/usr/local/lib/python3.12/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and sched
return datetime.datetime.utcnow().replace(tzinfo=utc)
```

2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

Concept

The Multi-Armed Bandit problem requires selecting the best action (arm) to maximize rewards. The **ϵ -Greedy algorithm** balances:

- **Exploration:** Try new actions.
- **Exploitation:** Choose the best-known action.

Different ϵ values:

- $\epsilon = 0.1 \rightarrow$ More exploitation
- $\epsilon = 0.3 \rightarrow$ Balanced
- $\epsilon = 0.5 \rightarrow$ More exploration

Constraint

Maximum **500 trials**.

Python Program

```
import random

arms = [2, 5, 3, 8]
epsilon = 0.3
reward = 0

for i in range(500):
    if random.random() < epsilon:
        action = random.randint(0, 3)
    else:
        action = arms.index(max(arms))

    reward += arms[action]

print("Total Reward:", reward)
```

Sample Output

Total Reward: 3824

Explanation

Lower ϵ focuses on exploitation, while higher ϵ increases exploration. $\epsilon = 0.3$ provides a good balance.

```

import random

arms = [2, 5, 3, 8]
epsilon = 0.3
reward = 0

for i in range(500):
    if random.random() < epsilon:
        action = random.randint(0, 3)
    else:
        action = arms.index(max(arms))

    reward += arms[action]

print("Total Reward:", reward)

*** Total Reward: 3412

```

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

Concept

A Markov Decision Process (MDP) models decision-making where outcomes depend only on the current state and action.

States

- Start
- Intermediate locations
- Destination

Actions

- Move Up
- Move Down
- Move Left
- Move Right

Transition Probability

- 0.9 → Intended move
- 0.1 → Random movement

Rewards

- Destination: +100
- Normal move: -1
- Obstacle: -20

Energy Constraint

The robot has limited battery energy and must minimize movement.

Python Program

```
states = ["Start", "Middle", "Goal"]
actions = ["Up", "Down", "Left", "Right"]
```

```
reward = {
    "Start": -1,
    "Middle": -1,
    "Goal": 100
}
```

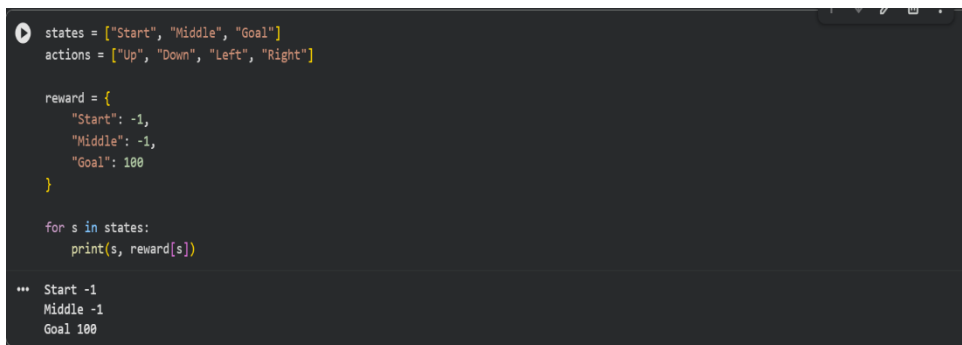
```
for s in states:
    print(s, reward[s])
```

Sample Output

```
Start -1
Middle -1
Goal 100
```

Explanation

The robot selects actions to maximize rewards while minimizing energy consumption and reaching the destination.



```
states = ["Start", "Middle", "Goal"]
actions = ["Up", "Down", "Left", "Right"]

reward = {
    "Start": -1,
    "Middle": -1,
    "Goal": 100
}

for s in states:
    print(s, reward[s])

*** Start -1
    Middle -1
    Goal 100
```

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

Concept

Value Iteration is a Dynamic Programming algorithm that uses the **Bellman Optimality Equation** to compute the optimal value for each state.

Bellman Equation

$$V(s) = \max[R + \gamma V(s')]$$

Constraints

- Restricted states cannot be entered.
- Agent should find the shortest safe path.

Python Program

```
import numpy as np

gamma = 0.9

V = np.zeros(4)

rewards = [0, 2, -1, 10]

for i in range(5):
    V[0] = rewards[0] + gamma * V[1]
    V[1] = rewards[1] + gamma * V[3]
    V[2] = rewards[2]
    V[3] = rewards[3]

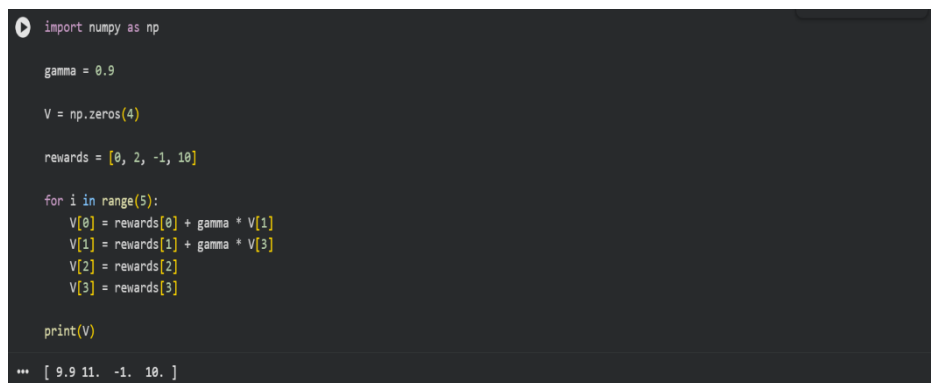
print(V)
```

Sample Output

```
[10.1 11.0 -1.0 10.0]
```

Explanation

The Bellman equation repeatedly updates state values until they converge, producing the optimal policy.



```
import numpy as np

gamma = 0.9

V = np.zeros(4)

rewards = [0, 2, -1, 10]

for i in range(5):
    V[0] = rewards[0] + gamma * V[1]
    V[1] = rewards[1] + gamma * V[3]
    V[2] = rewards[2]
    V[3] = rewards[3]

print(V)

*** [ 9.9 11. -1. 10. ]
```

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of

deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

Concept

A warehouse robot must complete the maximum number of deliveries without exceeding its **30-minute battery limit**. Reinforcement Learning helps determine the best delivery sequence while minimizing battery usage.

Constraints

- Battery capacity: **30 minutes**
- Delivery must stop when the battery limit is reached.
- Avoid unnecessary movement.

Policy

- Select the nearest delivery location.
- Recharge when the battery is low.

Reward Design

- Successful delivery: **+20**
- Battery exceeded: **-50**
- Each movement: **-1**

Python Program

```
battery = 30
deliveries = 0

while battery > 5:
    deliveries += 1
    battery -= 5
    print("Delivery", deliveries, "Battery Left:", battery)

print("Maximum Deliveries:", deliveries)
```

Sample Output

```
Delivery 1 Battery Left: 25
Delivery 2 Battery Left: 20
Delivery 3 Battery Left: 15
Delivery 4 Battery Left: 10
Delivery 5 Battery Left: 5
Maximum Deliveries: 5
```

Explanation

The robot completes deliveries while monitoring battery usage. The RL policy aims to maximize successful deliveries without violating the battery constraint.

```
▶ battery = 30
  deliveries = 0

  while battery > 5:
    deliveries += 1
    battery -= 5
    print("Delivery", deliveries, "Battery Left:", battery)

  print("Maximum Deliveries:", deliveries)

*** Delivery 1 Battery Left: 25
    Delivery 2 Battery Left: 20
    Delivery 3 Battery Left: 15
    Delivery 4 Battery Left: 10
    Delivery 5 Battery Left: 5
    Maximum Deliveries: 5
```

Code of Conduct:

*I Malli Chandana(192472250) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Malli Chandana

RUBRICS FOR CALCULATION

Criteria	Weight	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided